

Modélisation non paramétrique par des fonctions B-spline

Cindy Lançon, Clara Gambardello, Karla Pem

2025-11-27

Import des librairies

```
library(HRW)
library(dplyr)
library(mgcv)
library(splines)
library(knitr)
library(ggplot2)

data(WarsawApts)

appart<-WarsawApts
```

Nous travaillons sur un jeu de données concernant les prix de ventes des appartements de Varsovie en Pologne. On cherche à prédire la variable “areaPerMzloty”.

Les différentes variables sont :

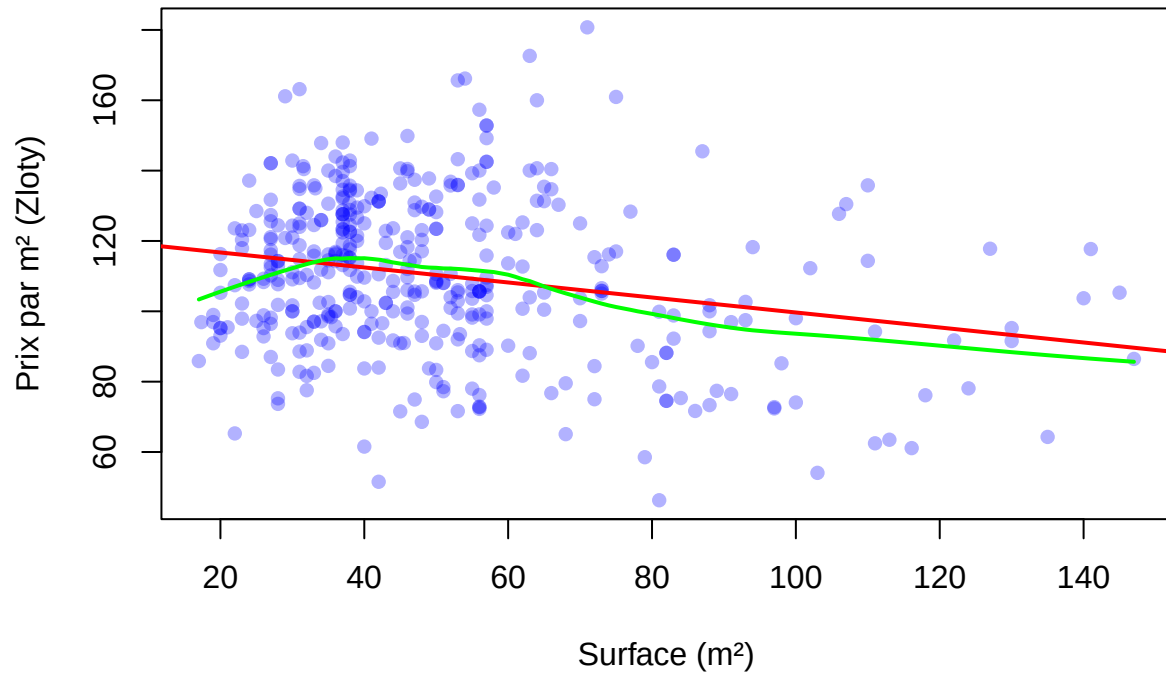
- areaPerMzloty : le prix au m².
- construction.date : date de construction de l’appartement.
- surface : surface de l’appartement.
- n.rooms : le nombre de chambre dans l’appartement.
- floors : l’étage de l’appartement.

On se propose dans un premier temps de représenter le prix au mètre carré en fonction des autres variables quantitatives afin de visualiser si on observe ou non des relations linéaires ou non.

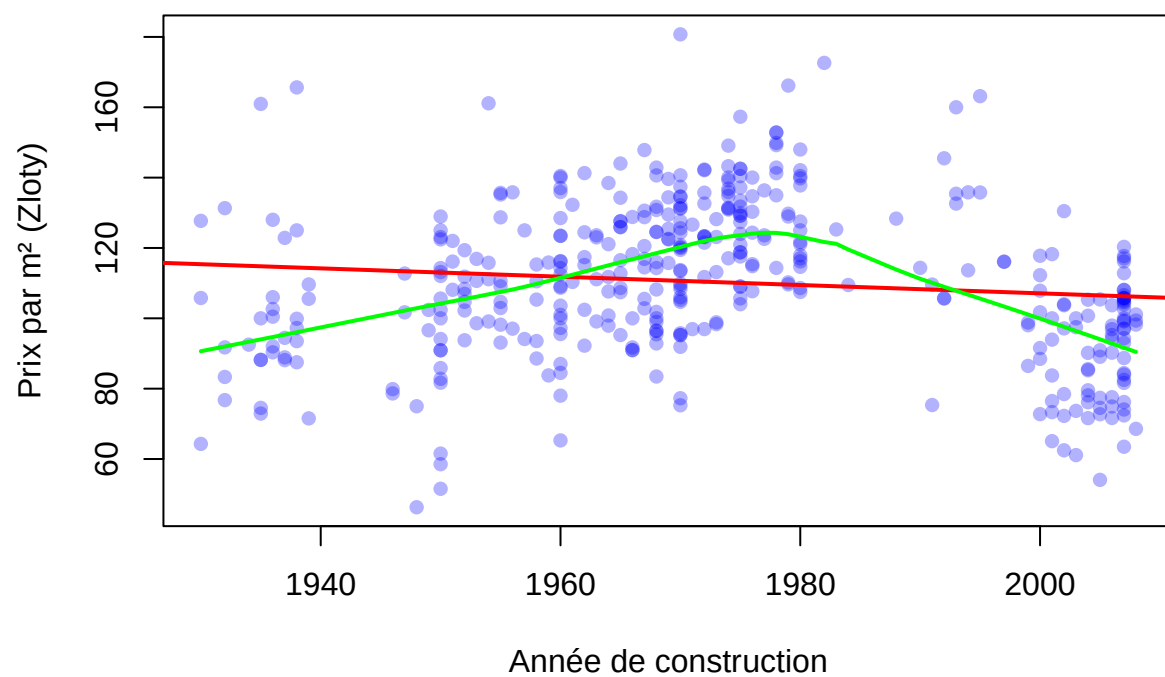
Partie 1 : Méthodes univariés

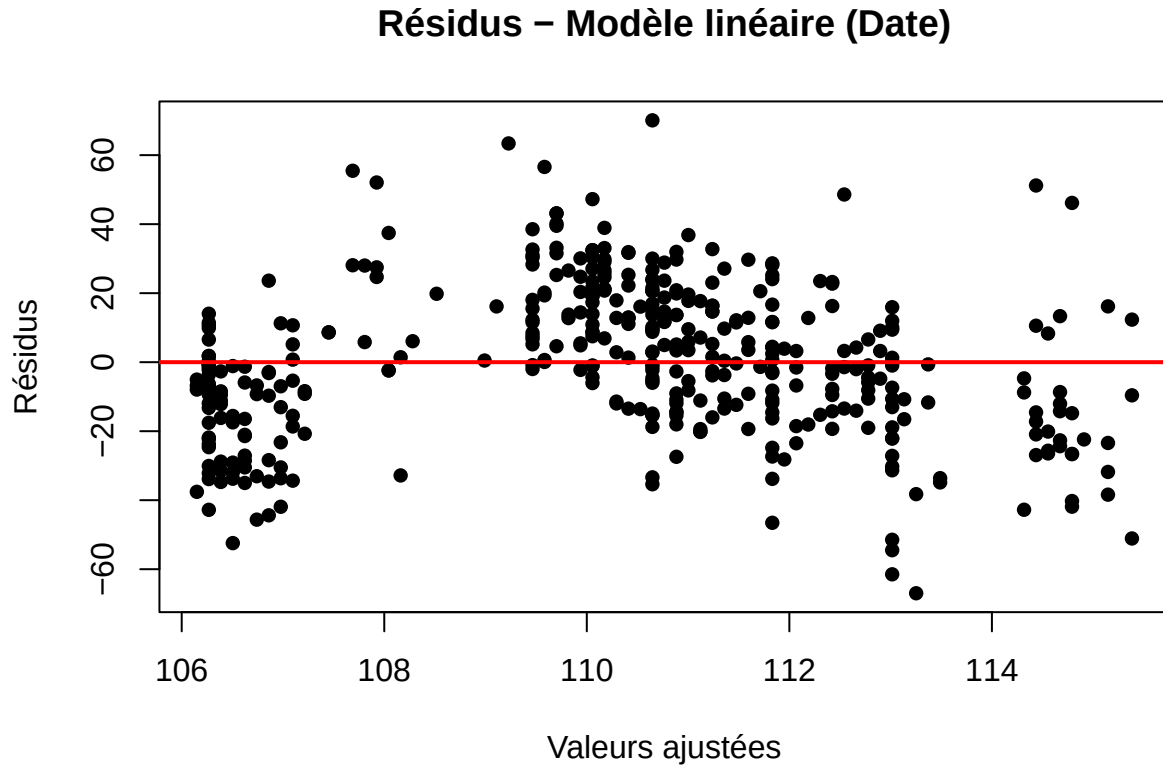
On commence par tracer différents graphique pour choisir la variable que nous utiliserons dans nos modèles univariés, on trace également la droite de régression linéaire ainsi qu’une courbe d’ajustement non-linéaire.

Prix vs Surface



Prix vs Année de construction





On observe les graphiques de surface et de construction date, ce que l'on peut constater c'est que l'on observe une tendance avec une légère forme de cloche sur la variable construction date alors que la surface apparaît comme un nuage de point sans tendance particulière.

Nous choisissons d'utiliser des méthodes non paramétriques, en effet nous constatons que le graphique des résidus du modèle linéaire montre une tendance en cloche, les résidus ne sont pas bien réparti, c'est une façon de voir que le modèle linéaire n'est pas adapté.

Modèle avec utilisation de B-splines Les B-splines constituent une méthode d'estimation non paramétrique efficace pour modéliser des relations non linéaires : elles permettent d'approximer une fonction inconnue par une combinaison linéaire de fonctions de base polynomiales définies par morceaux. Les paramètres modifiables sont le degré des polynômes et K nombre de noeud qui est lié au nombre de degrés.

Les fonctions B-splines $B_1(\cdot), \dots, B_q(\cdot)$ avec $\text{dimension} = \text{nbnoeuds} + m = q = K + m$, qui s'écrivent comme des combinaisons linéaires de polynômes tronqués de degré $m - 1$:

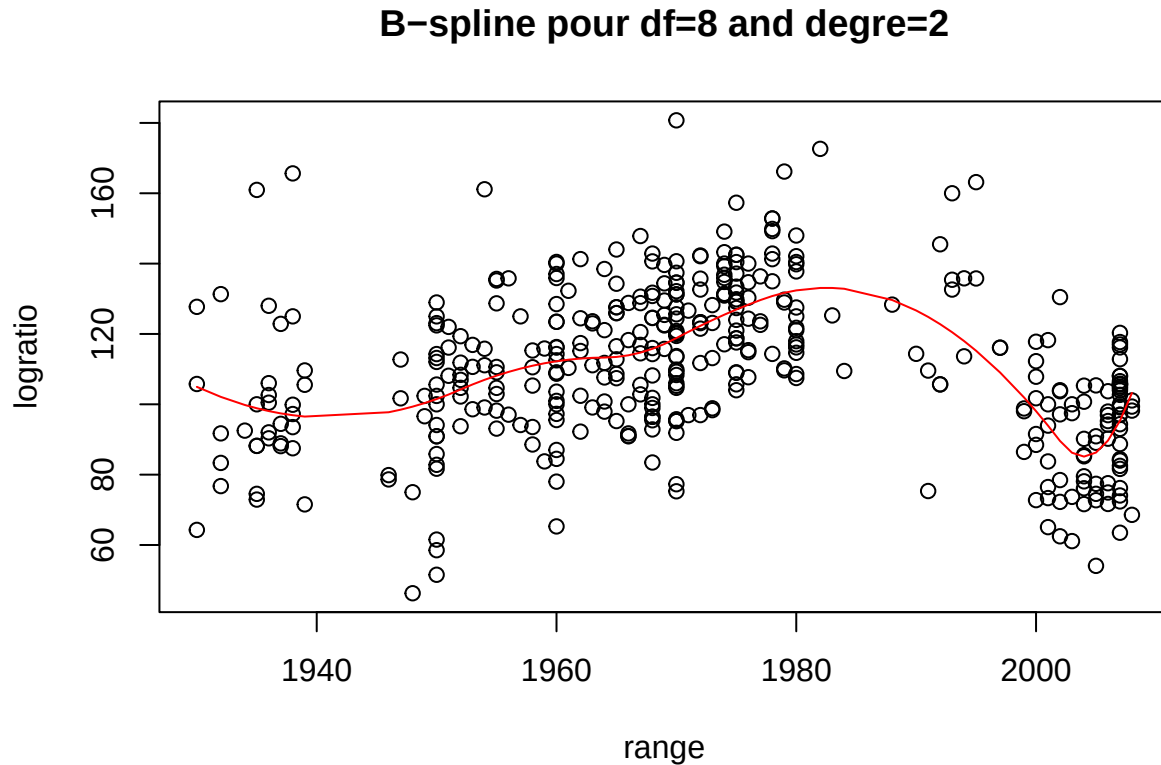
$$B_j = (\xi_j - \xi_{j-m}) \sum_{l=0}^m \frac{(\xi_{j-l} - x)_+^{m-1}}{\prod_{r=0, l \neq r}^m (\xi_{j-l} - \xi_{j-r})}$$

B_j est non-nulle uniquement sur les intervalles entre les noeuds $\xi_{j-m}, \dots, \xi_j$.

Le modèle s'écrit alors :

$$y_i = \sum_{j=1}^q \beta_j B_j(x_i) + \epsilon_i$$

Nous commençons par estimer un modèle de régression non paramétrique utilisant des B-splines en choisissant de manière arbitraire le degré 2 avec 8 degrés de liberté, le modèle est ajusté via la fonction `lm()` avec la base spline générée par `bs()` :



La représentation graphique de l'ajustement montre que les B-splines permettent de modéliser une tendance non linéaire cohérente entre l'année de construction et le prix au mètre carré.

Afin de perfectionner ce modèle on cherche K le nombre de noeud par validation croisée, avec K dans l'intervall [3, 20].

```
K_grid <- seq(3, 20, by=1)
```

```
CV_spline <- function(K, data, nfold=5, degree=2) {  
  n <- nrow(data)  
  folds <- cut(seq(1,n), breaks=nfold, labels=FALSE)  
  err <- rep(0, nfold)  
  for (i in 1:nfold) {  
    test_id <- which(folds == i)  
    train_id <- which(folds != i)  
    train <- data[train_id, ]  
    test <- data[test_id, ]  
  }  
}
```

```

mod <- lm(areaPerMzloty ~ bs(construction.date, df=K+degree+1, degree=degree, intercept=TRUE) - 1,
          data=train)

ypred <- predict(mod, newdata=test)

err[i] <- mean((test$areaPerMzloty - ypred)^2)
}

return(mean(err))
}

```

```

CV_errors_2 <- sapply(K_grid, CV_spline, data=apart, degree=2)
best_K_2 <- K_grid[which.min(CV_errors_2)]

paste("Erreur minimal", min(CV_errors_2))

```

```
## [1] "Erreur minimal 341.680107705241"
```

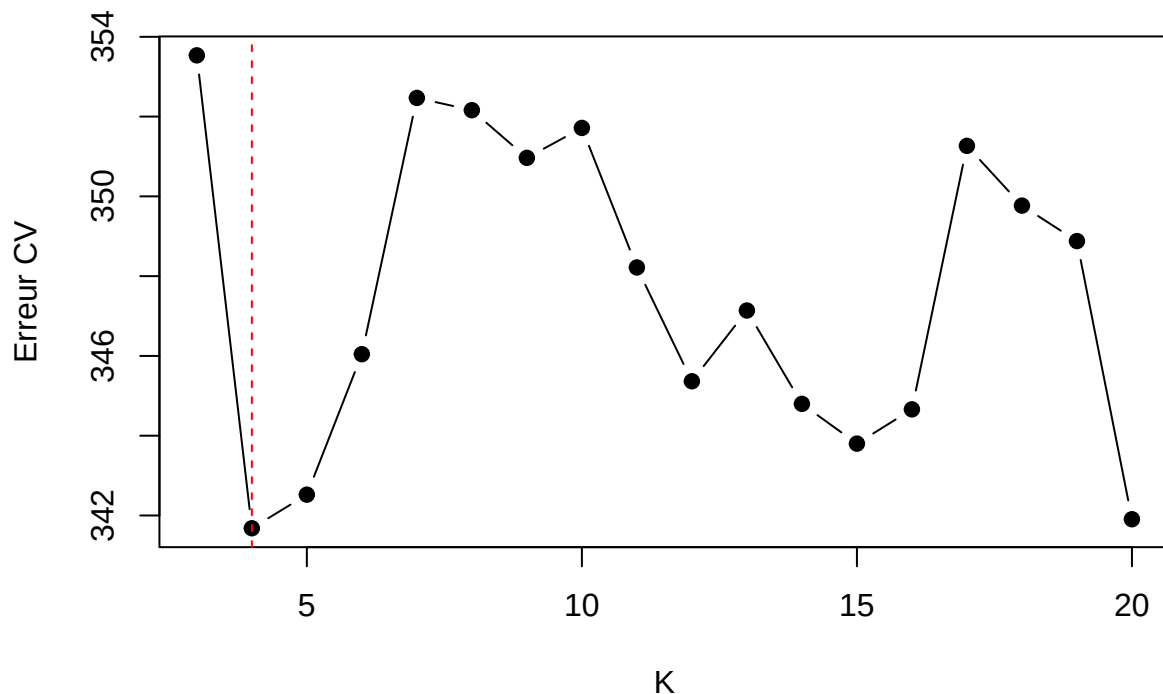
```
paste("K optimal : ",best_K_2)
```

```
## [1] "K optimal : 4"
```

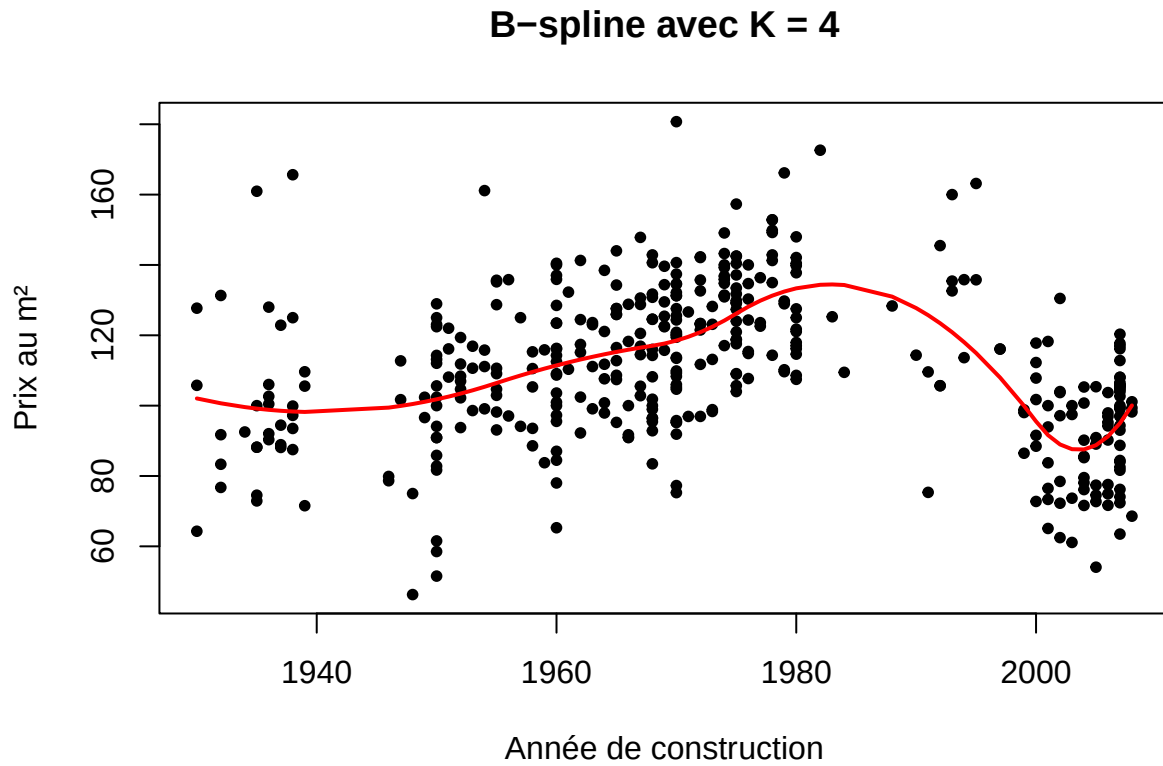
On obtient l'erreur la plus faible pour $K=4$. On essaie également de changer le degré des polynômes, mais c'est avec des splines quadratiques que l'erreur est la plus faible.

On représente l'erreur en fonction de K pour visualiser l'endroit où l'erreur est minimale

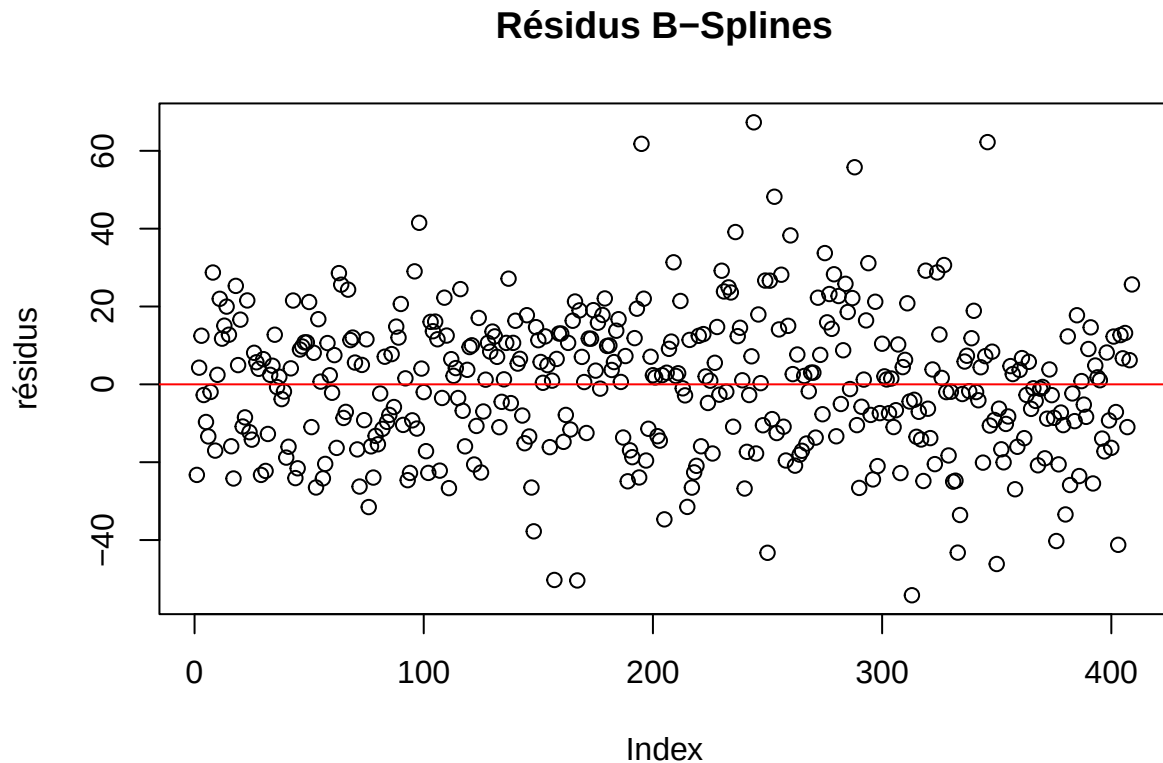
Validation croisée pour choisir K



On retrouve bien $K=4$. On refait donc un modèle avec ce nouveau paramètre optimisé :



On obtient une courbe qui capture globalement bien la tendance des données même si elle ne paraît pas forcément très régulière. On trace les résidus afin de vérifier leur distribution :



En observant les résidus on constate qu'ils sont bien répartis autour de 0. Le modèle est bien adapté.

On calcule l'erreur quadratique moyenne ainsi que l'AIC.

Comparaison des modèles On essaie également de pénaliser nos splines sur cette même base B-splines pour observer les résultats, en effet la pénalisation permet un meilleur lissage en gardant une bonne estimation.

On a directement optimisé K par validation croisée à 5 couches, pour K entre 5 et 35 et on a obtenu une erreur minimale pour $K=20$ beaucoup plus important que sans pénalisation, en traçant le graphique on s'aperçoit que la courbe est beaucoup moins régulière pour une MSE et un AIC quasiment similaire à celui sans pénalisation avec moins de noeud. On choisit donc de conserver le modèle le plus simple

On constate également que sur des splines pénalisés la base choisie, B-splines ou polynômes tronqués ne modifie pas de beaucoup la MSE et l'AIC.

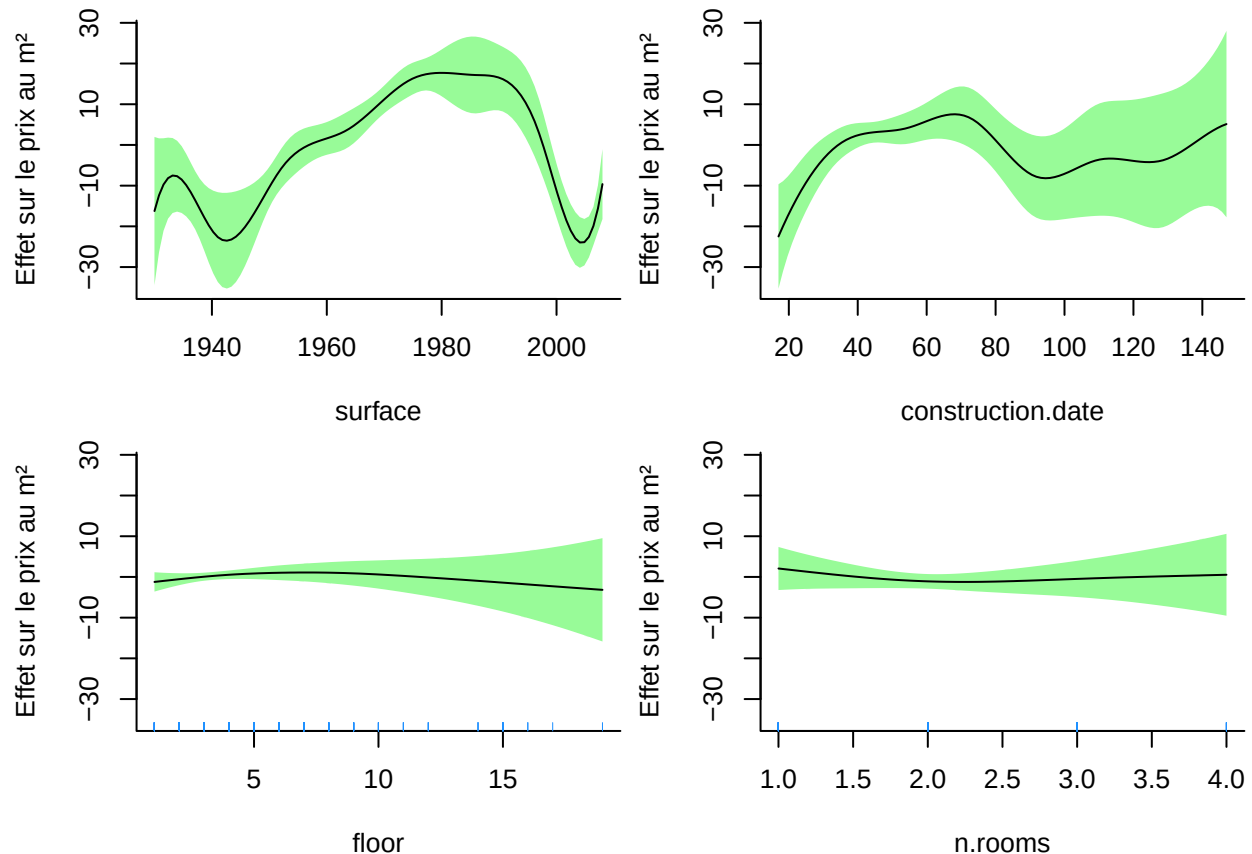
Table 1: Comparaison des performances des modèles univariés

Modèle	MSE	AIC
Régression polynomiale	338.8	3555.3
P-spline (B-splines)	338.7	3521.3
P-spline (p. tronqués)	339.8	3521.3
B-spline	316.4	3531.4

Partie 2 Méthodes multivariées

On passe maintenant aux modèles multivariés, en optimisant k (dimension de la base) par validation croisée. Le paramètre de lissage λ (sp dans $s()$) est optimisé directement dans la fonction `gam()` avec la validation croisée généralisée.

On analyse l'effet de chaque prédicteur sur la variable à prédire, on s'aperçoit que pour les variable nombre de pièces et étage, la relation est constante. Après des essais de modèles semi-paramétriques linéaire en ces deux variables et non linéaire pour les autres, on s'aperçoit que ce ne sont pas les modèles avec les plus faibles MSE ou AIC on se concentre donc sur un modèle avec surface et date de construction.



Nous réalisons un modèle avec P-splines avec paramètre `bs` par défaut de R qui correspond aux “Thin plate regression splines”, on effectue une validation croisée à 5 couches pour choisir le paramètre k , la dimension de la base.

```
cv_k_pspline_multi_r <- function(K, data, n_folds=5) {
  n <- nrow(data)
  folds <- cut(seq(1,n),breaks=n_folds, labels = FALSE)
  mse_folds <- numeric(n_folds)
  for (i in 1:n_folds) {
    test_idx <- which(folds == i)
    train_data <- data[-test_idx, ]
    test_data <- data[test_idx, ]

    #Modèle
    mod <- gam(areaPerMzloty ~ s(construction.date, k=K) + s(surface, k=K), data = train_data)
```

```

    pred <- predict(mod, newdata = test_data)
    mse_folds[i] <- mean((test_data$areaPerMzloty - pred)^2)
  }
  return(mean(mse_folds))
}

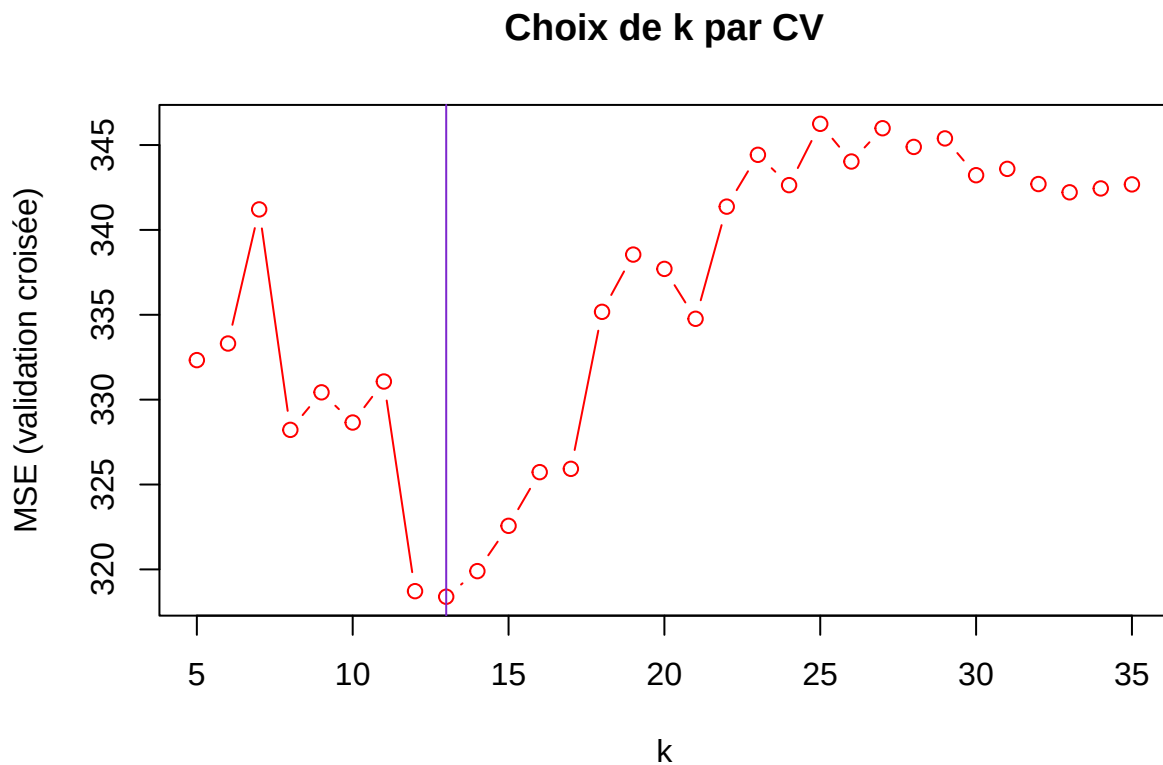
#Grille de K à tester
K_test <- seq(5,35, by=1)
mse_cv_multi <- sapply(K_test, cv_k_p spline_multi_r, data=WarsawApts, n_folds=5)

res_cv_pen_multi <- data.frame(
  K = K_test,
  mse_cv_multi = round(mse_cv_multi,3)
)

best_K_multi_p_r<-K_test[ which.min(mse_cv_multi)]

plot(K_test, mse_cv_multi, type="b", col="red",
     xlab = "k",
     ylab = "MSE (validation croisée)",
     main= "Choix de k par CV")
abline( v=best_K_multi_p_r, col="purple3")

```

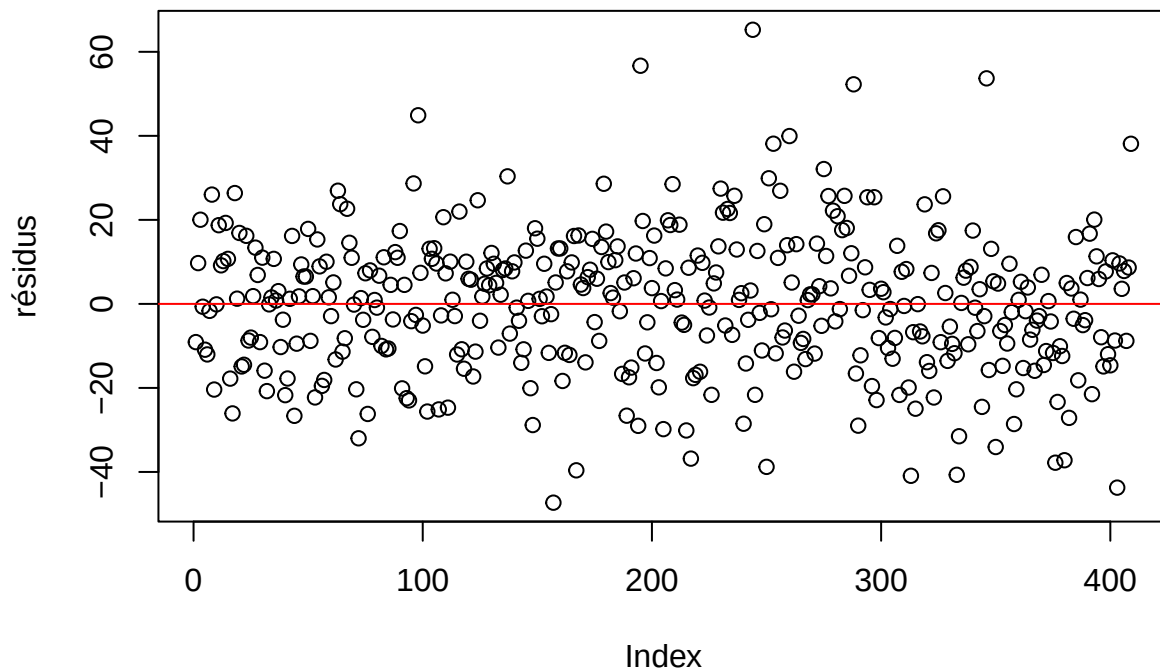


On obtient un k optimal égal à 13, on utilise ce paramètre dans notre modèle dont on affiche le résumé :

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## areaPerMzloty ~ s(construction.date, k = best_K_multi_p_r) +
##       s(surface, k = best_K_multi_p_r)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 110.2923    0.8368   131.8  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##             edf Ref.df      F  p-value
## s(construction.date) 11.449 11.924 17.988  < 2e-16 ***
## s(surface)           7.683  9.183  3.997 6.22e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.415   Deviance explained = 44.2%
## GCV = 301.23   Scale est. = 286.4       n = 409
```

On trace les résidus pour vérifier leur distribution :

Résidus modèle multivariés



On retrouve des résidus équitablement répartis autour de 0, ce modèle est bien adapté. Le modèle met en

évidence un effet significatif de la date de construction et de la surface sur le prix au m² ($p < 0,001$ pour les deux). Le modèle explique environ 44 % de la déviance.

Comparaison des modèles En essayant avec d'autres modèles nous obtenons des MSE et des AIC généralement plus élevés :

Table 2: Comparaison des performances des modèles multivariés

Modèle	MSE	AIC
Régression polynomiale	315.4	3530.0
B-splines	287.2	3507.6
P-spline (B-spline)	273.5	3500.2
P-spline(p.tronqués)	273.1	3499.8
P-spline (par défaut)	272.3	3496.2

Le modèle final est donc :

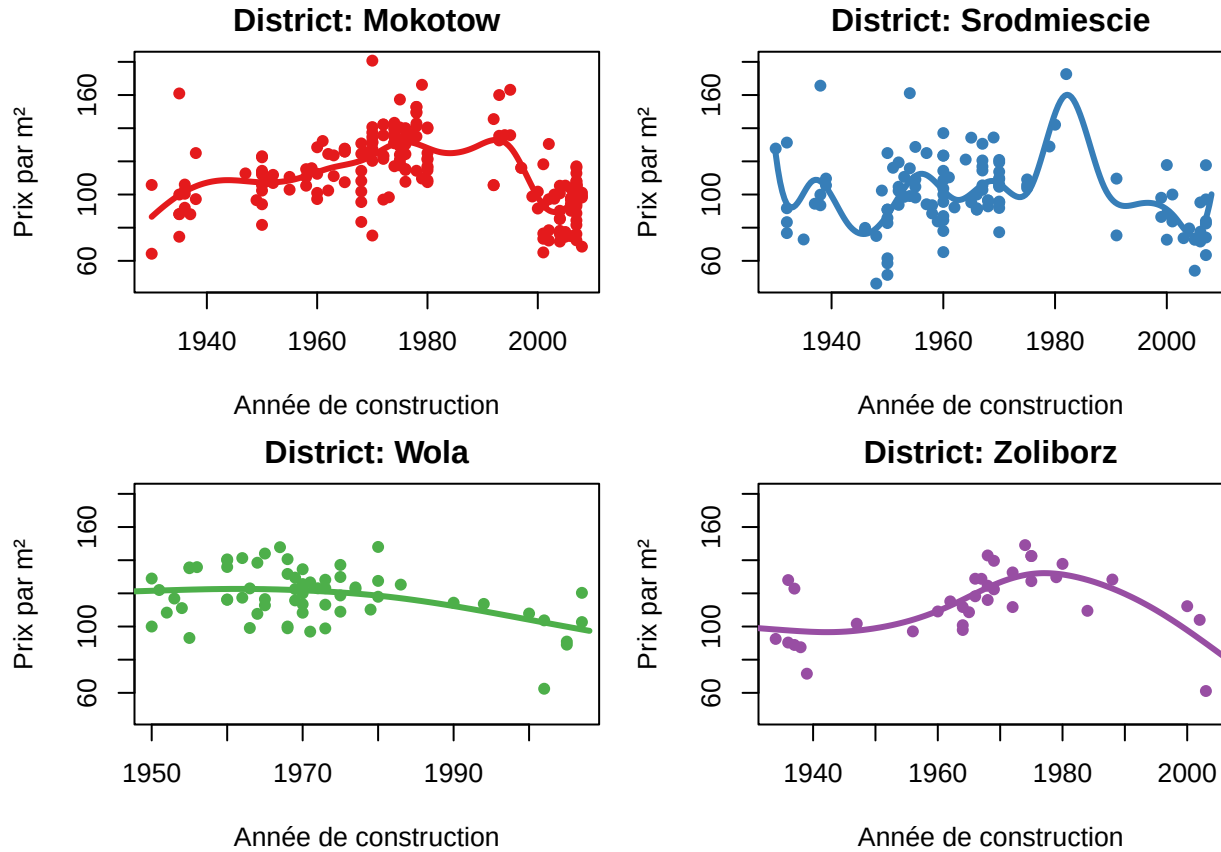
$$areaPerMzloty_i = \beta_0 + f_1(construction.date_i) + f_2(surface_i) + \epsilon_i$$

où f_1 et f_2 sont des fonctions (radiales) estimées dans une base de thin plate regression splines (tp : splines de plaque mince), qui utilise une pénalité sur les dérivées.

Partie 3 Méthode avec une variable qualitative et une variable quantitative

Le jeu de données contient une variable facteur qui peut être utilisée dans le modèle : le district prenant comme valeurs Mokotow, Srodimiescie, Wola, Solibroz.

On peut se poser la question : l'effet de l'année de construction est-il le même dans tous les districts ?



On observe que les courbes montrent des tendances différentes selon le district. L'effet de l'année de construction varie selon le quartier. Nous utilisons donc un modèle avec interaction (`by= district`) permettant d'estimer une courbe spécifique pour chaque quartier.

Choix de la base : Nous avons testé différentes bases de splines : B-spline (`bs = "bs"`), P-spline avec pénalité sur les différences (`bs="ps"`) et thin plate regression spline (par défaut).

Les base "ps", avec une pénalité sur la différence, obtient les meilleurs résultats.

Nous avons construit des modèles emboîtés de complexité croissante afin d'identifier le meilleur modèle.

- Modèle 1 : Interaction district x année de construction
- Modèle 2 : Modèle 1 et ajout de la variable surface.
- Modèle 3 : Modèle 2 et ajout de l'étage.
- Modèle 4 : Modèle 2 et ajout du nombre de pièces.

Pour définir le choix de la dimension de la base (k), nous avons décidé d'utiliser la règle de Ruppert et al. (2002), qui est la suivante :

$$K = \min(0.25 \times \text{nombre de valeurs uniques } x_i, 35)$$

```

K_constr <- min(0.25 * length(unique(WarsawApts$construction.date)), 35)
K_surface <- min(0.25 * length(unique(WarsawApts$surface)), 35)
K_floor <- min(0.25 * length(unique(WarsawApts$floor)), 35)
K_rooms <- min(0.25 * length(unique(WarsawApts$n.rooms)), 35)

degree <- 3
k_constr <- K_constr + degree + 1
k_surface <- K_surface + degree + 1
k_floor <- K_floor + degree + 1
k_rooms <- K_rooms + degree + 1

# k = K + degree +1
# K = k - degree -1

# Modèle 1 : district additif (effet constant)
mod1_b <- gam(areaPerMzloty ~ district
              + s(construction.date, by=district, k=k_constr, bs="ps"),
              data=WarsawApts)

#Mod 2 avec surf et construction
mod2_b <- gam(areaPerMzloty ~ district
              + s(construction.date, by=district, k=k_constr, bs="ps")+
              s(surface, k=k_surface, bs="ps"),
              data=WarsawApts)

# Modèle 3 : modèle avec surface et floor
mod3_b <- gam(areaPerMzloty ~ district +
              s(construction.date, by = district, k=k_constr, bs="ps") +
              s(surface, k=k_surface, bs="ps") +
              s(floor, k=k_floor, bs="ps"),
              data = WarsawApts)

# Modèle 4 : modèle complet (avec surface et floor et n.rooms)
mod4_b <- gam(areaPerMzloty ~ district +
              s(construction.date, by = district, k=k_constr, bs="ps") +
              s(surface, k=k_surface, bs="ps") +
              s(n.rooms, k=length(unique(WarsawApts$n.rooms)), bs="ps"),
              data = WarsawApts)

AIC(mod1_b, mod2_b, mod3_b, mod4_b)

##           df      AIC
## mod1_b 34.07240 3463.641
## mod2_b 66.62071 3414.280
## mod3_b 70.00302 3414.593
## mod4_b 69.01412 3411.967

```

Le modèle 1 obtient l'AIC la plus haute, tandis que le modèle 2, 3 et 4 présentent des AIC très proche.

Afin de déterminer statistiquement si l'ajout des variables améliore significativement l'ajustement, nous avons réalisé des test ANOVA entre nos modèles emboîtés.

On compare :

H_0 : Le modèle réduit est suffisant

H_1 : Le modèle complet apporte une amélioration significative

```
## Analysis of Deviance Table
##
## Model 1: areaPerMzloty ~ district + s(construction.date, by = district,
##   k = k_constr, bs = "ps")
## Model 2: areaPerMzloty ~ district + s(construction.date, by = district,
##   k = k_constr, bs = "ps") + s(surface, k = k_surface, bs = "ps")
##   Resid. Df Resid. Dev    Df Deviance      F      Pr(>F)
## 1      371.85      96547
## 2      338.14      72980 33.71      23567 3.2895 1.584e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
## Analysis of Deviance Table
##
## Model 1: areaPerMzloty ~ district + s(construction.date, by = district,
##   k = k_constr, bs = "ps") + s(surface, k = k_surface, bs = "ps")
## Model 2: areaPerMzloty ~ district + s(construction.date, by = district,
##   k = k_constr, bs = "ps") + s(surface, k = k_surface, bs = "ps") +
##   s(floor, k = k_floor, bs = "ps")
##   Resid. Df Resid. Dev    Df Deviance      F Pr(>F)
## 1      338.14      72980
## 2      334.76      71838 3.382      1142.2 1.5984 0.1837
```

```
## Analysis of Deviance Table
##
## Model 1: areaPerMzloty ~ district + s(construction.date, by = district,
##   k = k_constr, bs = "ps") + s(surface, k = k_surface, bs = "ps")
## Model 2: areaPerMzloty ~ district + s(construction.date, by = district,
##   k = k_constr, bs = "ps") + s(surface, k = k_surface, bs = "ps") +
##   s(n.rooms, k = length(unique(WarsawApts$n.rooms)), bs = "ps")
##   Resid. Df Resid. Dev    Df Deviance      F Pr(>F)
## 1      338.14      72980
## 2      336.11      71724 2.0254      1255.9 2.9481 0.05313 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Modèle 1 vs modèle 2 : Le test nous indique une p-value de $1.58e-08 < 0.05$, donc on peut conclure que l'ajout de la surface améliore très significativement le modèle. On va donc garder le modèle 2 avec la variable surface.

Modèle 2 vs modèle 3 : Le test nous indique une p-value de $0.1837 > 0.05$, donc on peut conclure que l'ajout de l'étage n'améliore pas significativement le modèle et n'apporte donc pas d'information supplémentaire. On garde donc le modèle 2.

Modèle 2 vs modèle 4 : Le test nous indique une p-value de $0.053 > 0.05$, donc on peut conclure que l'ajout du nombre de pièce n'améliore pas significativement malgré que la p-value soit proche du seuil.

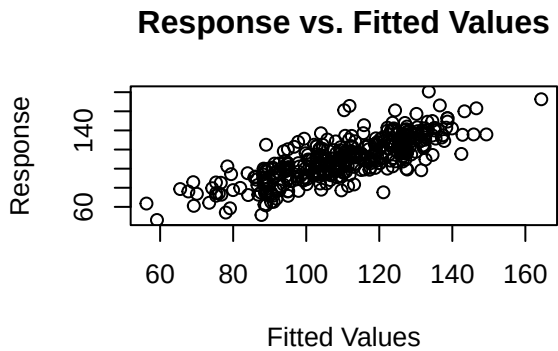
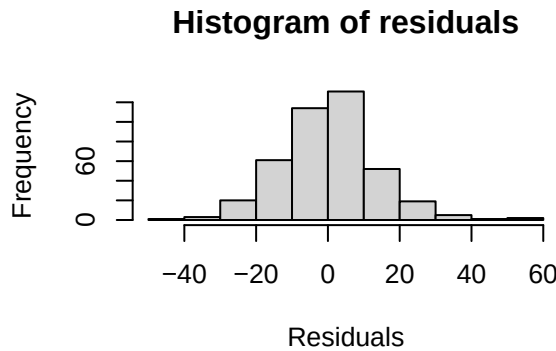
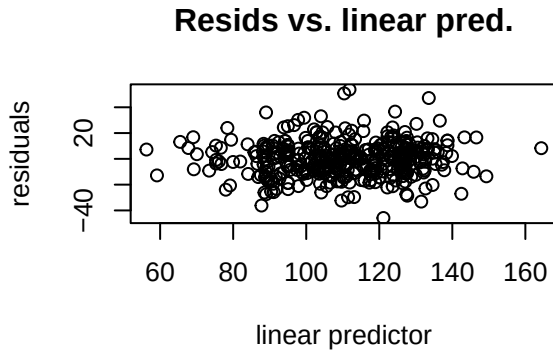
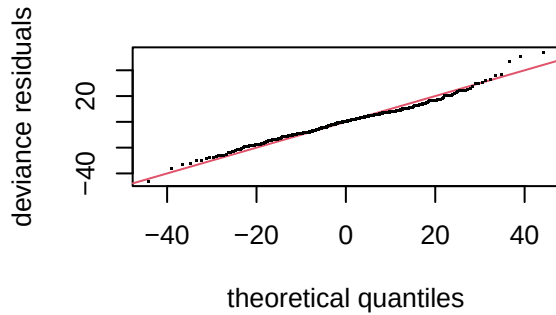
Choix du modèle final En se basant sur les résultats de l'AIC et des tests ANOVA, on choisit donc de retenir le modèle 2 comme le modèle final optimal.

Notre modèle est donc :

$$\text{areaPerMzloty}_i = \beta_0 + \sum_{j=1}^3 \beta_j \mathbf{1}_{\{d_i=j\}} + \sum_{j=0}^3 f_j(c_i) \mathbf{1}_{\{d_i=j\}} + g(s_i) + \varepsilon_i$$

$$\text{areaPerMzloty}_i = \begin{cases} \beta_0 + f_0(\text{construction.date}_i) + g(\text{surface}_i) + \varepsilon_i, & \text{si Mokotow,} \\ \beta_0 + \beta_1 + f_1(\text{construction.date}_i) + g(\text{surface}_i) + \varepsilon_i, & \text{si Srod miescie,} \\ \beta_0 + \beta_2 + f_2(\text{construction.date}_i) + g(\text{surface}_i) + \varepsilon_i, & \text{si Wola,} \\ \beta_0 + \beta_3 + f_3(\text{construction.date}_i) + g(\text{surface}_i) + \varepsilon_i, & \text{si Zoliborz.} \end{cases}$$

- β_0 = intercept de référence (pour Mokotow).
- β_j = effet fixe du quartier j.
- f_j = **fonction lisse spécifique au quartier j** pour l'effet de l'année de construction (interaction).
- g = **fonction lisse globale** pour l'effet de la surface (identique pour tous les quartiers).



```
##
## Method: GCV   Optimizer: magic
## Smoothing parameter selection converged after 22 iterations.
## The RMS GCV score gradient at convergence was 0.001209075 .
## The Hessian was positive definite.
## Model rank = 109 / 109
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##
```

	k'	edf	k-index	p-value
s(construction.date):districtMokotow	19.00	17.12	0.98	0.26
s(construction.date):districtSrodmiemie	19.00	14.09	0.98	0.34
s(construction.date):districtWola	19.00	2.20	0.98	0.34
s(construction.date):districtZoliborz	19.00	3.66	0.98	0.41
s(surface)	29.00	24.55	1.01	0.50

Validation du modèle :

- Le QQ-plot indique que les résidus suivent approximativement une distribution normale.
- On n'observe aucun pattern systématique, ce qui confirme l'homoscédasticité.
- L'histogramme des résidus montre une distribution plutôt symétrique centrée sur zéro.

```
# Créer une grille de prédiction
date_grid <- expand.grid(
  construction.date = seq(min(WarsawApts$construction.date),
                           max(WarsawApts$construction.date),
                           length.out = 100),
  district = levels(WarsawApts$district),
  surface = median(WarsawApts$surface)
)

# Prédiction
date_pred <- predict(mod2_b,
                     newdata = date_grid,
                     type = "response",
                     se.fit = TRUE)

date_plot_data <- date_grid %>%
  mutate(
    pred = date_pred$fit,
    se = date_pred$se.fit,
    lower = pred - 1.96 * se,
    upper = pred + 1.96 * se
  )

ggplot(date_plot_data, aes(x = construction.date, y = pred, color = district)) +
  geom_line(size = 1.2) +
  geom_ribbon(aes(ymin = lower, ymax = upper, fill = district),
            alpha = 0.2, color = NA) +
  labs(title = "Prix au m² en fonction de l'année de construction (à surface fixée)",
       subtitle = "Par district (avec intervalles de confiance à 95%)",
```

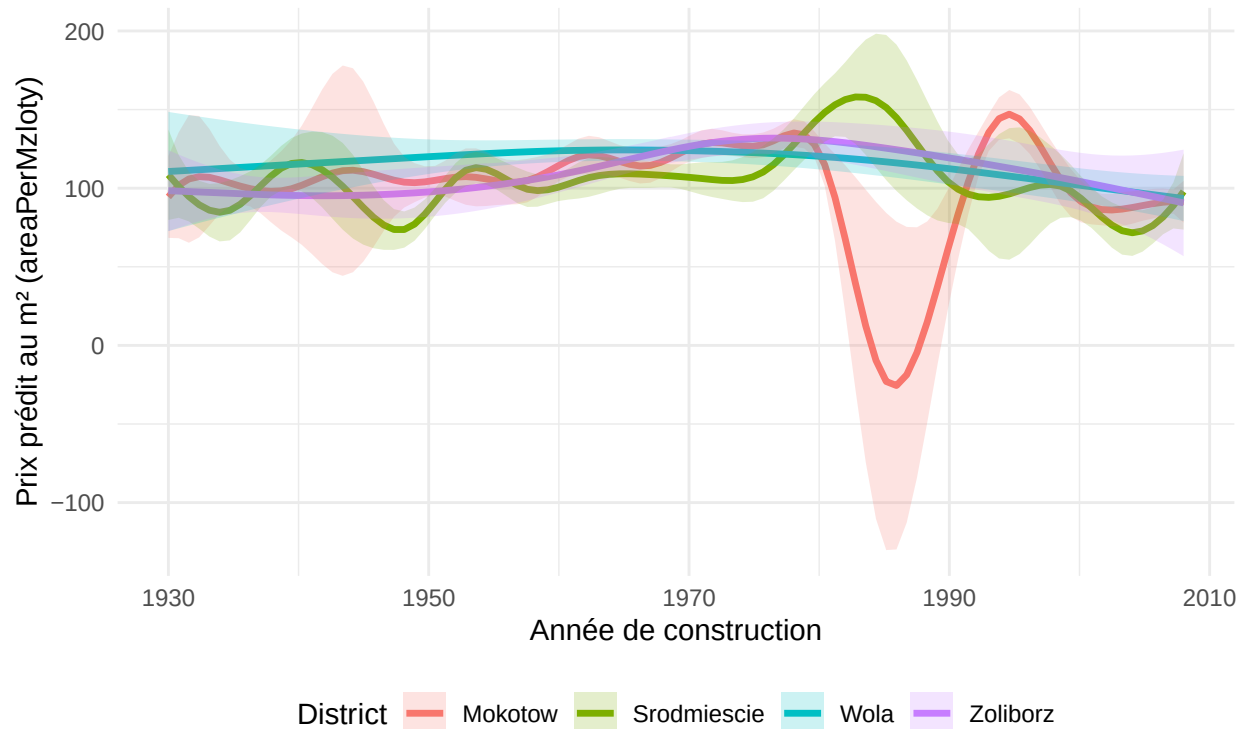
```

x = "Année de construction",
y = "Prix prédit au m² (areaPerMzloty)",
color = "District",
fill = "District") +
theme_minimal() +
theme(plot.title = element_text(face = "bold", size = 14),
      legend.position = "bottom")

```

Prix au m² en fonction de l'année de construction (à surface fixé)

Par district (avec intervalles de confiance à 95%)



Ce projet nous a permis d’explorer différentes approches de modélisation non paramétrique pour prédire le prix au m² des appartements.

Après avoir enrichi progressivement nos modèles en intégrant d’autres variables explicatives (surface, étage, nombre de pièces), le modèle finale sélectionné est le deuxième modèle de la partie variables qualitatives et quantitatives, il présente le plus petit AIC parmi tous les modèles traités dans ce projet. C’est un modèle qui contient une interaction entre le district et l’année de construction ainsi qu’un effet global de la surface. On utilise une base “ps” pour le paramètre “bs” qui est la base de polynômes tronqués avec une pénalité sur la différence.